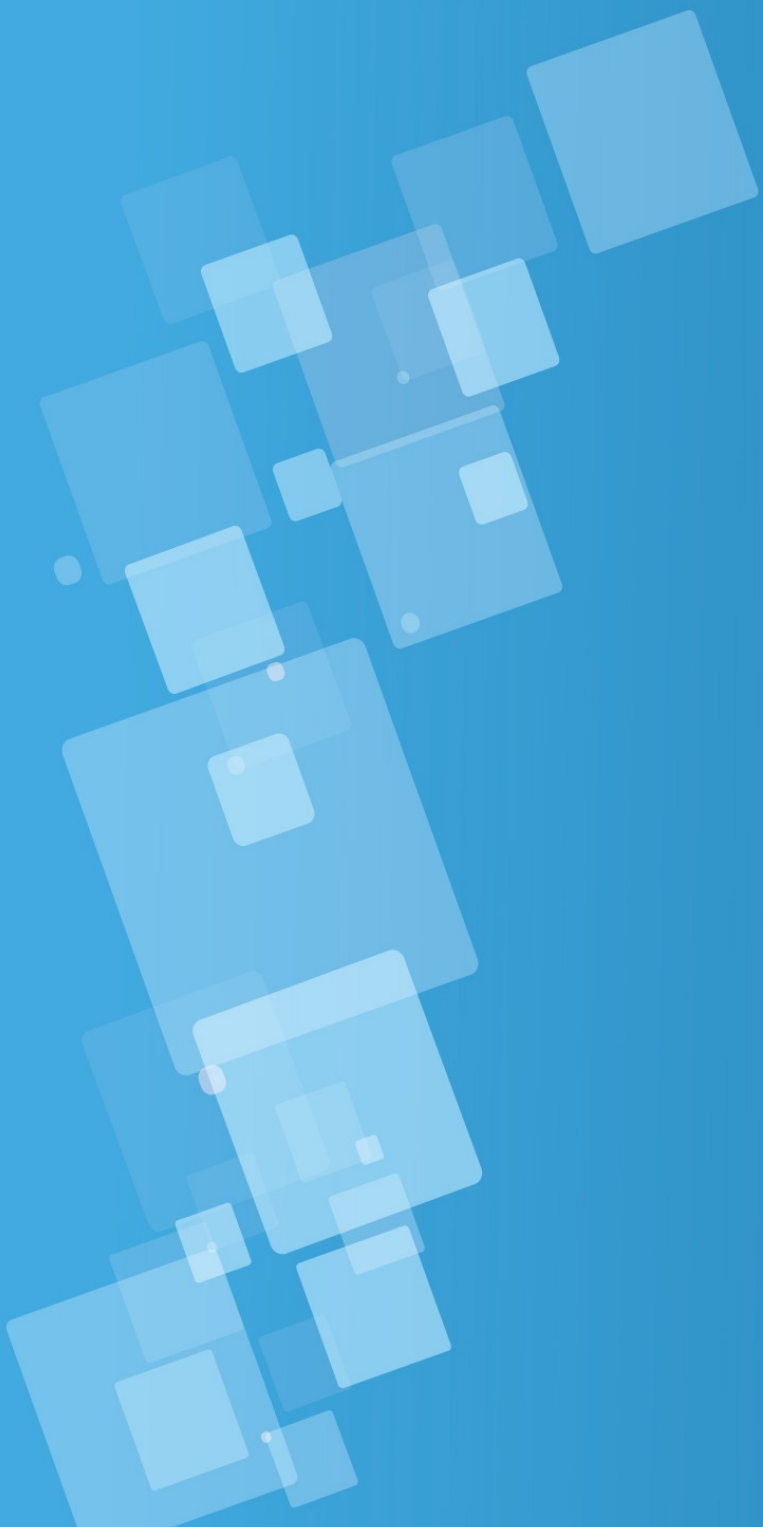




Técnicas Digitales II

Introducción a Microprocesadores

Dr.Ing.Steiner Guillermo

- Bibliografía

Computer Organization and Design ARM Edition, 2017
Capítulo 1.1 a 1.8





Introducción

- Más allá de las revoluciones agrícola e industrial, la acelerada evolución del hardware ha dado lugar a la revolución de la información: un cambio de paradigma donde la computación redefine cada aspecto de la sociedad moderna.



Introducción

- Gordon Moore predijo en 1965 que la densidad de transistores se duplicaría cada 18-24 meses.
- Al reducir el tamaño, disminuye el costo por transistor y aumenta la velocidad de conmutación.
- Consecuencia: tecnologías que hace 20 años eran "ciencia ficción" hoy son aplicaciones de consumo masivo.



Introducción

- **Automóviles:** Electrónica para control de inyección (eficiencia) y seguridad activa (ABS, airbags).
- **Celulares:** Reducción de costos que permitió el acceso a comunicaciones inalámbricas a escala global.
- **Genoma Humano:** Potencia de cálculo que hizo viable el mapeo genético a costos razonables.
- **World Wide Web:** Digitalización del conocimiento, reemplazando a bibliotecas y medios físicos.
- **Motores de Búsqueda:** Herramientas fundamentales para localizar información en el mar de datos de Internet.



Clasificación de la Computación Moderna

- **Computadores Personales (PC):** Optimización de la relación precio/rendimiento para un único usuario.
- **Servidores / Cloud:** Orientado a cargas considerables o muchas tareas pequeñas, se prioriza además la fiabilidad. Abarca, de servidores pequeños similares a una PC a supercomputadoras.
- **Sistemas Embebidos:**
 - Es la clase de computadoras mas grande en volumen.
 - Están diseñadas para ejecutar una sola aplicación o un conjunto muy específico de tareas.
 - Se encuentran integradas a otros dispositivos.
 - El software se considera una parte integral y fija del hardware..



Sistemas Embebidos

Es la categoría mas diversa y exigente de la industria:

- **Restricciones de Diseño Extremas:** Es el factor más crítico es el costo, muchos sistemas embebidos se fabrican por millones, por lo que centavos impacta en la viabilidad del producto.
- **Consumo de Energía:** Fundamental para dispositivos alimentados por batería y para evitar el sobrecalentamiento sin usar ventiladores.
- **Confiabilidad y Disponibilidad:** Baja tolerancia a falla. En dispositivos de consumo (como un televisor) puede significar una desconfianza hacia la marca, pero en operaciones críticas (frenos ABS, aviónica, medicina) puede significar riesgo de vida .
- **Interacción con el Mundo Real:** A menudo deben procesar señales en tiempo real, respondiendo a estímulos externos (sensores) en intervalos de tiempo garantizados, algo que no es prioritario en una PC.

Términos de almacenamiento

- ¿ Cuánto es un KB ?
- Los terminos KB o TB se referían a 1024 o 2^{40} respectivamente
- Pero para desarrolladores de almacenamiento secundario y para el sistema internacional (SI) KB es 1000 y TB es 10^{12}
- Para evitar ambigüedades en 1998 se crea nuevos prefijos que se detallan en la IEC 80000-13

T.Decimal	Abr.	Valor	T.Binario	Abr.	Valor	% Mayor
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%



La Era Post-PC y los PMD

- Personal Mobile Devices (PMD): El smartphone ha reemplazado a la PC como el dispositivo principal.
- Requisitos de los PMD:
 - Eficiencia energética extrema para durar todo el día.
 - Interfaces nuevas (pantallas táctiles, acelerómetros).
 - Conectividad inalámbrica constante.



Cloud Computing y Warehouse Scale Computers

- Los dispositivos móviles (PMD) se apoyan en la "Nube" para tareas pesadas.
 - Warehouse Scale Computers (WSC) Centros de datos masivos que actúan como un único computador gigante, (ej Google, Amazon, Meta)
 - Software como Servicio (SaaS): Aplicaciones que no corren localmente sino en servidores remotos (ej. motores de búsqueda Microsoft 365, Google Drive, Zoom, etc).



Las 8 Grandes Ideas de la Arquitectura

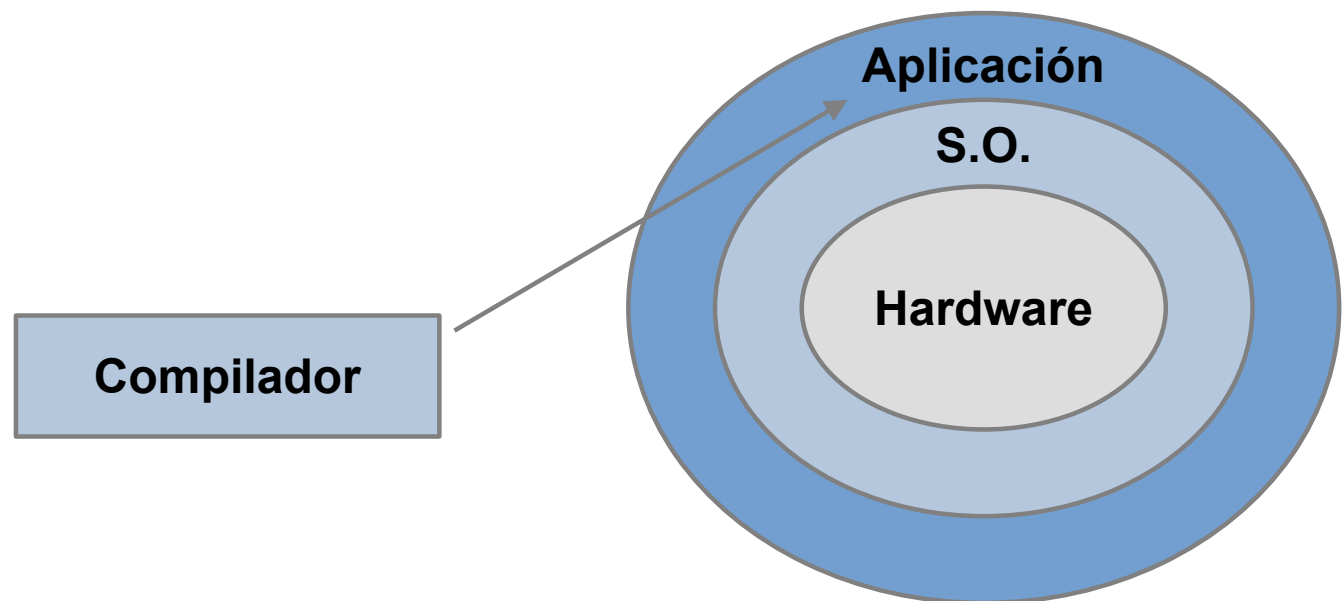
Ocho grandes ideas que los arquitectos informáticos han inventado en los últimos 60

- Diseño para la Ley de Moore.
- Uso de Abstracción para simplificar el diseño.
- Hacer el caso común mas rápido.
- Rendimiento vía Paralelismo.
- Rendimiento vía Pipelining.
- Rendimiento vía Predicción.
- Jerarquía de Memorias.
- Dependencia vía Redundancia.

Capas de Software y Abstracción

El software se organiza en niveles para ocultar los detalles técnicos del hardware a los programadores de aplicaciones:

- **Software de Aplicación:** La capa más externa (ej. procesadores de texto, bases de datos).
- **Software de Sistema:** Proporciona servicios comunes y actúa como intermediario entre la aplicación y el hardware.
 - **Sistema Operativo (OS):** Gestiona los recursos.
 - **Compilador**
- **Hardware**





Del Lenguaje de Alto Nivel al Hardware

La traducción ocurre en varios pasos:

- **Lenguaje de Alto Nivel** (ej. C, Java): Utiliza palabras y notación algebraica, permitiendo que el programa sea portátil e independiente de la máquina.
- **Lenguaje Ensamblador**: Una representación simbólica de las instrucciones de la máquina (ej. ADD A, B) que un programa llamado ensamblador convierte a binario.
- **Lenguaje de Máquina (Binario)**: La forma final que entiende el hardware, compuesta exclusivamente por bits (0 y 1).



¿Por qué es vital en esta jerarquía?

Sin el compilador, la jerarquía de abstracción se rompería por dos razones:

- **Complejidad:** Los programadores tendrían que escribir millones de líneas en binario (0 y 1), lo cual es humanamente imposible para sistemas modernos.
- **Independencia del Hardware:** El compilador permite que el mismo programa de "Alto Nivel" pueda correr en una PC o en un sistema embebido. El compilador es el que sabe "hablar" el dialecto específico de cada hardware, traduciendo la misma intención a diferentes lenguajes de máquina.

El Flujo de Traducción Completo

```
swap(int v[], int k) {  
    int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

Compilador

swap:

```
LSL X10, X1, 3  
ADD X10, X0, X10  
LDUR X9, [X10, 0]  
LDUR X11, [X10, 8]  
STUR X11, [X10, 0]  
STUR X9, [X10, 8]  
BR LR
```

Ensamblador

```
0000 0000 1010 0010 0000 0001 0001 1000  
0000 0000 1000 0010 0001 0000 0010 0001  
1000 1101 1110 0010 0000 0000 0000 0000  
1000 1110 0001 0010 0000 0000 0000 0100  
1010 1110 0001 0010 0000 0000 0000 0000  
1010 1101 1110 0010 0000 0000 0000 0100  
0000 0011 1110 0000 0000 0000 0000 1000
```

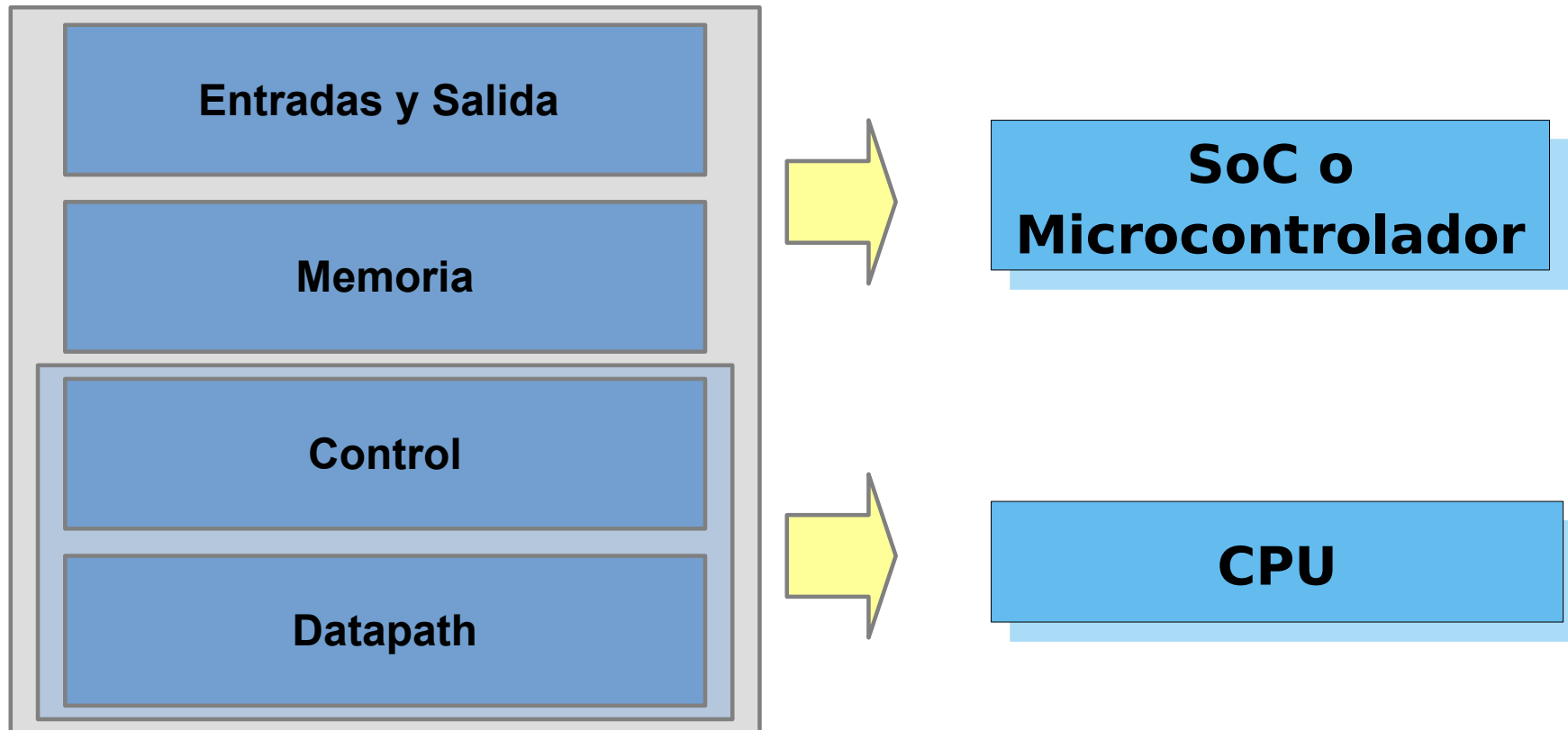


Hardware

Cualquier computadora, sin importar su escala, se puede dividir en cinco partes fundamentales:

- **Entrada (Input):** Cómo el computador recibe datos del mundo exterior (ej. micrófonos, sensores, pantallas táctiles).
- **Salida (Output):** Cómo devuelve los resultados al usuario o al entorno (ej. pantallas, actuadores, altavoces).
- **Memoria (Memory):** Donde se almacenan los programas en ejecución y los datos que estos necesitan.
- **Ruta de Datos (Datapath):** Realiza las operaciones aritméticas y lógicas.
- **Control:** La unidad que "comanda" a la ruta de datos, la memoria y los dispositivos de E/S según las instrucciones del programa.

Hardware





Hardware – Jerarquía de Memorias

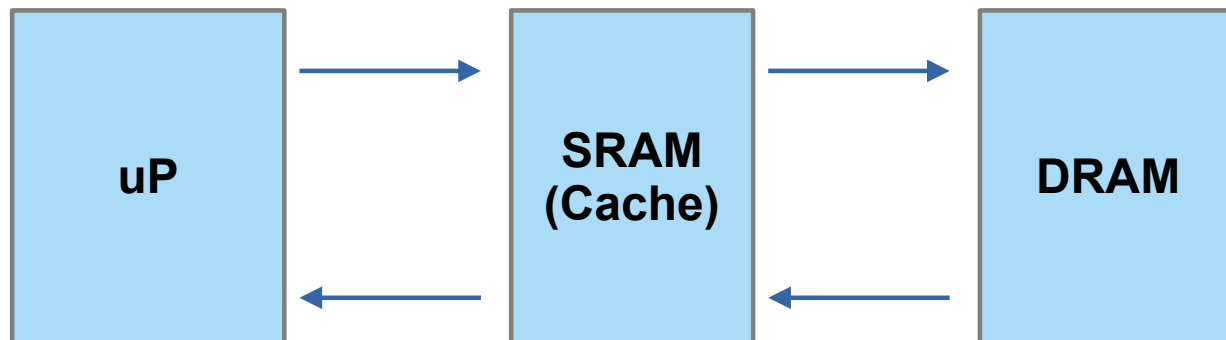
Actualmente, hay una diferencia muy grande de velocidad entre el procesador y la memoria principal (DRAM). La Caché soluciona este cuello de botella.

- **Memoria Caché (SRAM - Static RAM):**
 - Velocidad: Frecuencias cercanas a las del núcleo del procesador.
 - Función: Buffer de alta velocidad, almacenando datos recurrente.
 - Costo: Baja Densidad, lo que la hace cara.
- **Memoria Principal (DRAM - Dynamic RAM):**
 - Capacidad: Memoria de alta densidad; bajo costo por gigabytes.
 - Limitación: Mayor latencia debido a la necesidad de ciclos de refresco constantes.

Hardware – Jerarquía de Memorias

Arquitectura de Niveles:

- **SRAM (Cache):** Pequeña, rápida y cercana al CPU para compensar la lentitud de la memoria externa.
- **DRAM (Principal):** Grande y económica para contener el grueso del programa y los datos.





Hardware – Almacenamiento Secundario

Almacenamiento Secundario (No Volátil)

Retiene datos sin energía.

- **Memoria Flash:** Más rápida, sin partes móviles (usada en embebidos y SSD).
 - **Discos Magnéticos:** Más capacidad, más lentos (usados en servidores).
-
- Es la base de la jerarquía de memoria.
 - Se diferencia de la memoria convencional en que su gestión técnica pertenece al ecosistema de Entrada/Salida



Hardware - La Interfaz Hardware/Software (ISA)

La Arquitectura del Conjunto de Instrucciones o ISA: Interfaz abstracta entre el hardware y el software de más bajo nivel que engloba toda la información necesaria para escribir un programa en lenguaje de máquina que funcione correctamente, incluyendo instrucciones, registros, acceso a memoria, E/S, etcétera.

Permite una Independencia de la Implementación

Diferentes implementaciones (diseños físicos del hardware) pueden ejecutar el mismo software, siempre que cumplan con la misma ISA.



Arquitecturas mas conocidas

Con el paso de los años existieron muchas arquitecturas, la mayoría quedaron obsoletas, ejemplos como DEC, PowerPC o la prueba fallida de Intel con IA-64.

Otras aún persisten pero en nichos muy específicos como el Z80 o SPARC.

Muchas empresas como TI o Motorola, encontraron muy costoso mantener su arquitectura propia y migraron en su mayoría a ARM

Actualmente sobreviven un conjunto reducido de arquitecturas, de las cuales se sigue trabajando.



Arquitecturas mas conocidas

arm

1. ARM (Advanced RISC Machines)

Arquitectura principal en el mundo de los SE y dispositivos móviles. Se basa en una filosofía RISC (Reduced Instruction Set Computer).

Diseñada originalmente para computadoras personales (Acorn), hoy domina gracias a su muy buena eficiencia energética.

ARM no vende microprocesadores, vende licencias, eso permitió su difusión masiva.

Actualmente se dividen en:

- Cortex-A (aplicaciones potentes como celulares)
- Cortex-R (tiempo real crítico)
- Cortex-M (microcontroladores).

Usa un set de instrucciones de longitud fija (o comprimidas como Thumb-2) que facilita la segmentación (pipelining).

Arquitecturas mas conocidas



2. x86 / x64 (Intel & AMD)

La arquitectura estándar para PCs, laptops y servidores de alto rendimiento. Es una arquitectura CISC (Complex Instruction Set Computer).

Con una historia de más de 40 años, comenzando con el Intel 8086 en la primera PC.

Su mayor éxito fue la retrocompatibilidad (un programa de hace 30 años puede correr en un procesador actual).

Aunque es internamente compleja, las implementaciones modernas traducen las instrucciones CISC a "micro-operaciones" parecidas a RISC para ganar velocidad.

Arquitecturas mas conocidas



3. RISC-V

La gran promesa de código abierto que está revolucionando la industria.

Nacida en la Universidad de Berkeley (David Patterson), es una ISA abierta y libre de regalías.

Su diseño es modular: tiene un núcleo básico muy pequeño y se le pueden añadir extensiones según la necesidad (punto flotante, compresión, etc.).

Está ganando terreno rápidamente en microcontroladores y aceleradores de IA por su flexibilidad total y costo nulo de licencias.



Arquitecturas mas conocidas



4. MIPS (Microprocessor without Interlocked Pipelined Stages)

La arquitectura académica por excelencia.

Es la expresión más pura de la filosofía RISC. Aunque hoy es menos común en computadoras de consumo, sigue presente en routers, módems y consolas famosas como la PlayStation 1 y Nintendo 64.

Su estructura es tan limpia que es ideal para enseñar cómo funciona un procesador por dentro.



Arquitecturas mas conocidas



5. AVR (Microchip / Atmel)

La arquitectura detrás del famoso Arduino Uno.

Es una arquitectura de 8 bits (aunque con registros de 16) de tipo Harvard, diseñada para ser eficiente en ejecución de código C en dispositivos de muy bajo costo y pocos pines.

Su simplicidad la hace perfecta para el aprendizaje inicial en electrónica, aunque se queda corta frente a ARM cuando la complejidad del sistema crece.



Hardware - La Interfaz Hardware/Software (ISA) y ABI

Además, y para que el software sea realmente portátil entre sistemas similares, se introduce la (Interfaz Binaria de Aplicación o ABI)

- Es la combinación de la ISA más las interfaces proporcionadas por el software de sistema (como las llamadas al sistema operativo, como llamar a funciones, pase de parámetros, tipos de datos, etc).
- En el caso particular de los sistemas embebido se llama EABI, un estándar que simplifica algunos criterios para disminuir los requerimientos de memoria.

Estas reglas que implementa ABI o EABI, permite por ejemplo utilizar librerías precompiladas en nuestro proyecto,



Definiendo el Rendimiento

El rendimiento o *performance* puede ser definido de diferentes maneras

Para una computadora personal, mejor rendimiento significa menor:

- Tiempo de Respuesta o Tiempo de Ejecución: Cuánto tarda en completarse una tarea única.

Para un servidor, mejor rendimiento significa mayor:

- Ancho de Banda o Tasa de transferencia: Cuántas tareas se completan en un tiempo determinado.
- Mejorar el procesador → mejora el tiempo de respuesta y mejora el ancho de banda (esto último es cierto si no estamos al límite de la demanda).
- Aumentar la cantidad de procesadores → solo mejora el ancho de banda.



Medición del Rendimiento

Si simplificamos y analizamos el rendimiento solo como tiempo de respuesta, para una computadora X tenemos:

$$\text{Rendimiento}_X = \frac{1}{\text{tiempo de ejecución}_X}$$

Si comparamos dos computadoras X y Y

$$\text{Rendimiento}_X > \text{Rendimiento}_Y$$

$$\frac{1}{\text{tiempo de ejecución}_X} > \frac{1}{\text{tiempo de ejecución}_Y}$$

$$\text{tiempo de ejecución}_X < \text{tiempo de ejecución}_Y$$

El tiempo de ejecución de Y es mayor a X, si X es mas rápido que Y



Medición del Rendimiento

Si la computadora X es n veces más rápido que Y, entonces

$$\frac{\text{tiempo de ejecución}_Y}{\text{tiempo de ejecución}_X} = \frac{\text{Rendimiento}_X}{\text{Rendimiento}_Y} = n$$

El tiempo entonces es una medida del rendimiento de una computadora: la computadora que realiza la misma cantidad de trabajo en menos tiempo es la más rápida.



Tiempo de Reloj vs. Tiempo de CPU

Es importante distinguir qué tiempo estamos midiendo exactamente:

- **Tiempo transcurrido** (Wall-clock time o Elapsed time): Es el tiempo total que tarda una tarea en completarse, incluyendo accesos a disco, memoria, actividades de entrada/salida y el tiempo que el procesador dedica a otros programas (multitarea).
- **Tiempo de CPU**: Es el tiempo que el procesador dedica exclusivamente a ejecutar las instrucciones de un programa específico. No incluye el tiempo de espera por E/S o el tiempo dedicado a otros programas.



Desglose del Tiempo de CPU

El tiempo de CPU se divide a su vez en dos categorías:

- **Tiempo de CPU de usuario:** El tiempo invertido en ejecutar el código del programa propiamente dicho.
- **Tiempo de CPU del sistema:** El tiempo que el sistema operativo dedica a realizar tareas a petición del programa (como gestionar recursos).



Ciclos de Reloj (The Clock)

Casi todas las computadoras modernas se construyen utilizando un reloj de hardware que oscila a una frecuencia constante. Esto se denominan ciclos de reloj o ticks.

- **Período de reloj (Clock cycle time):** Es la duración de un ciclo completo (por ejemplo, 250 picosegundos).
- **Frecuencia de reloj (Clock rate):** Es el inverso del período (por ejemplo, 4 GHz).



La Ecuación del Rendimiento

La fórmula básica que permite calcular el tiempo de ejecución en función del hardware:

$$\text{Tiempo de CPU} = \text{Ciclos de reloj para un programa} \times \text{Tiempo de Ciclo de Reloj}$$

Si obtenemos los ciclos de reloj necesarios a partir de las instrucciones

$$\text{Tiempo de CPU} = \text{Instrucciones del programa} \times CPI \times \text{Tiempo de Ciclo de Reloj}$$

CPI (*Cycles Per Instruction*): Promedio de ciclos por instrucción.

- Esta métrica es la mas básica, solo utiliza el Tiempo de CPU del usuario, pero es una idea del impacto del software en el sistema

Comparación de segmentos de código

Ejemplo

Tenemos una CPU con 3 clases de instrucciones con los siguiente:
CPI A de 1 CPI, B de 2 CPI y C de 3 CPI

Luego dos secuencias de código con la siguientes instrucciones por categoría

- Código 1 = 2 de tipo A, 1 de tipo B y 2 de tipo C total 5 instruc.
- Código 2 = 4 de tipo A, 1 de tipo B y 1 de tipo C total 6 instruc.

¿ Cual es la mas rápida ?

- Código 1 = $(2 \times 1) + (1 \times 2) + (2 \times 3) = 10$ ciclos
- Código 2 = $(2 \times 4) + (1 \times 2) + (1 \times 3) = 9$ ciclos

$$CPI_1 = 10/5 = 2,0$$

$$CPI_2 = 9/6 = 1,5$$



Factores que modifican el rendimiento

Ya sabemos qué piezas componen el rendimiento. Ahora, la pregunta es:

¿Qué podemos modificar nosotros para que estos números cambien?

Componente	Que afecta	De que forma
Algoritmo	Instrucciones Posiblemente CPI	Determina la cantidad de instrucciones Puede determinar la complejidad de las instrucciones
Lenguaje de Programación	Instrucciones CPI	Las sentencias se traducen en instrucciones El lenguaje también puede afectar el CPI debido a sus características (mas abstracción requiere instrucciones con un CPI más alto).
Compilador	Instrucciones CPI	La eficiencia del compilador afecta tanto el número de instrucciones como el CPI
ISA	Instrucciones Clock CPI	La arquitectura del conjunto de instrucciones (ISA) afecta los tres aspectos,



El Muro de Potencia

La frecuencia de reloj como la potencia aumentaron rápidamente durante décadas y luego se estancaron en el 2004.

La razón: hemos alcanzado el límite práctico de potencia para la refrigeración de microprocesadores convencionales.

"Muro de Potencia" es el punto donde el calor generado por un microprocesador no puede ser disipado de forma eficiente o económica.

En CMOS el consumo de energía se produce en las transiciones

$$E \approx \frac{1}{2} \cdot C \cdot V^2$$

V = tensión, al bajarla la potencia se reduce de manera cuadrática

C = capacidad de carga en cada transistor



El Muro de Potencia

Finalmente la potencia requerida por un transistor, será el producto de la energía de una transición y la frecuencia.

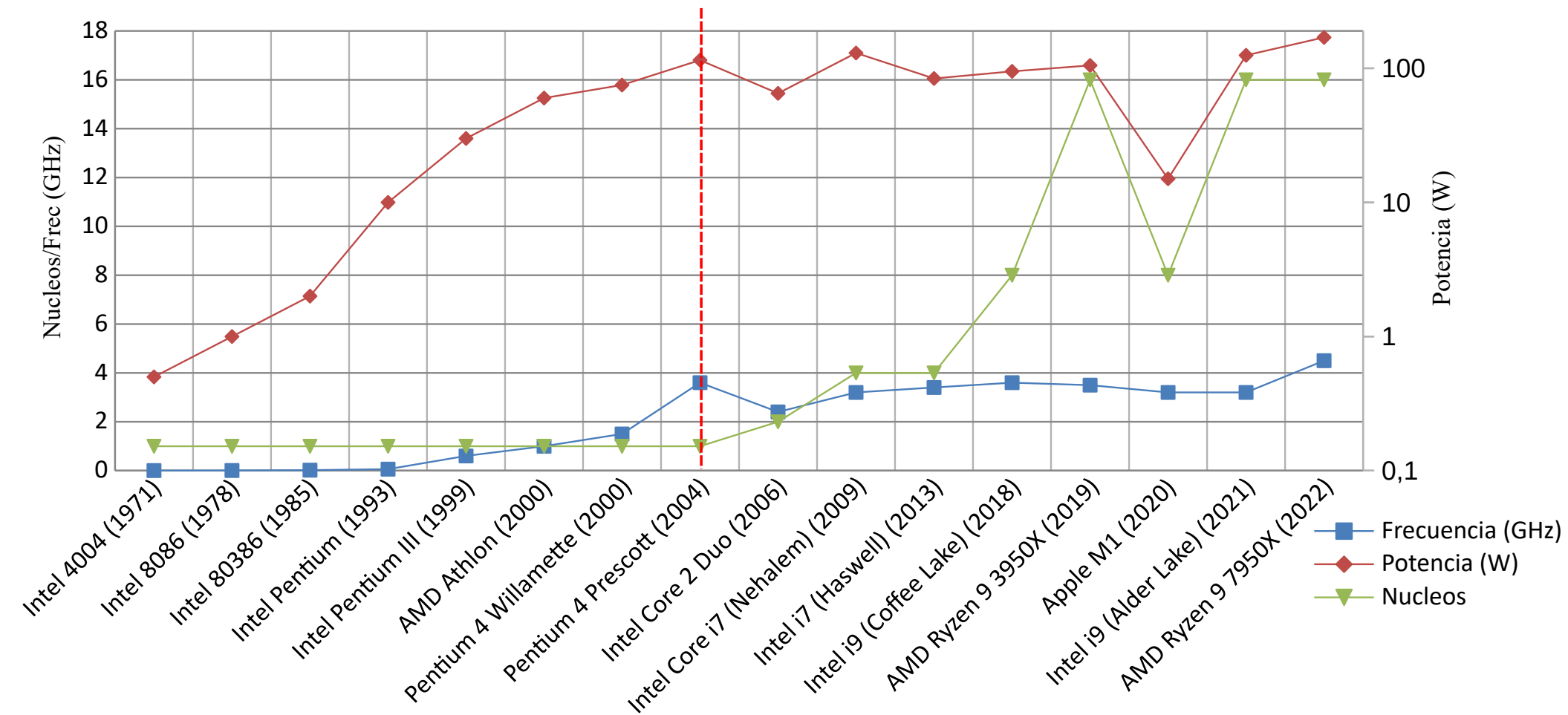
$$P_{\text{Dinámica}} \approx \frac{1}{2} \cdot C \cdot V^2 \cdot f$$

Durante los primeros años se reducía la tensión mientras que se aumentaba la frecuencia y complejidad del microprocesador.

Pasamos de 5V a 1V, permitiendo que la potencia solo aumente 30 veces mientras que la frecuencia aumentó 1000 veces.

El Muro de Potencia

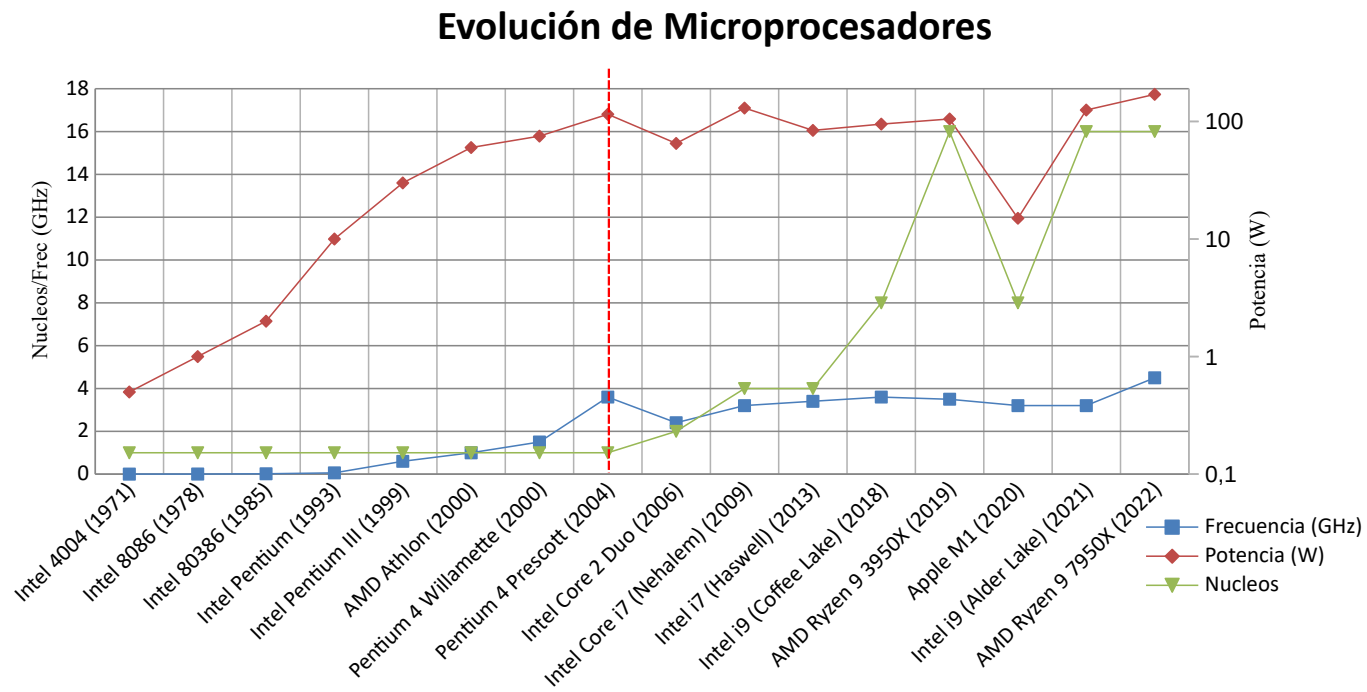
Evolución de Microprocesadores



El Muro de Potencia

Esto funcionó hasta el 2004.

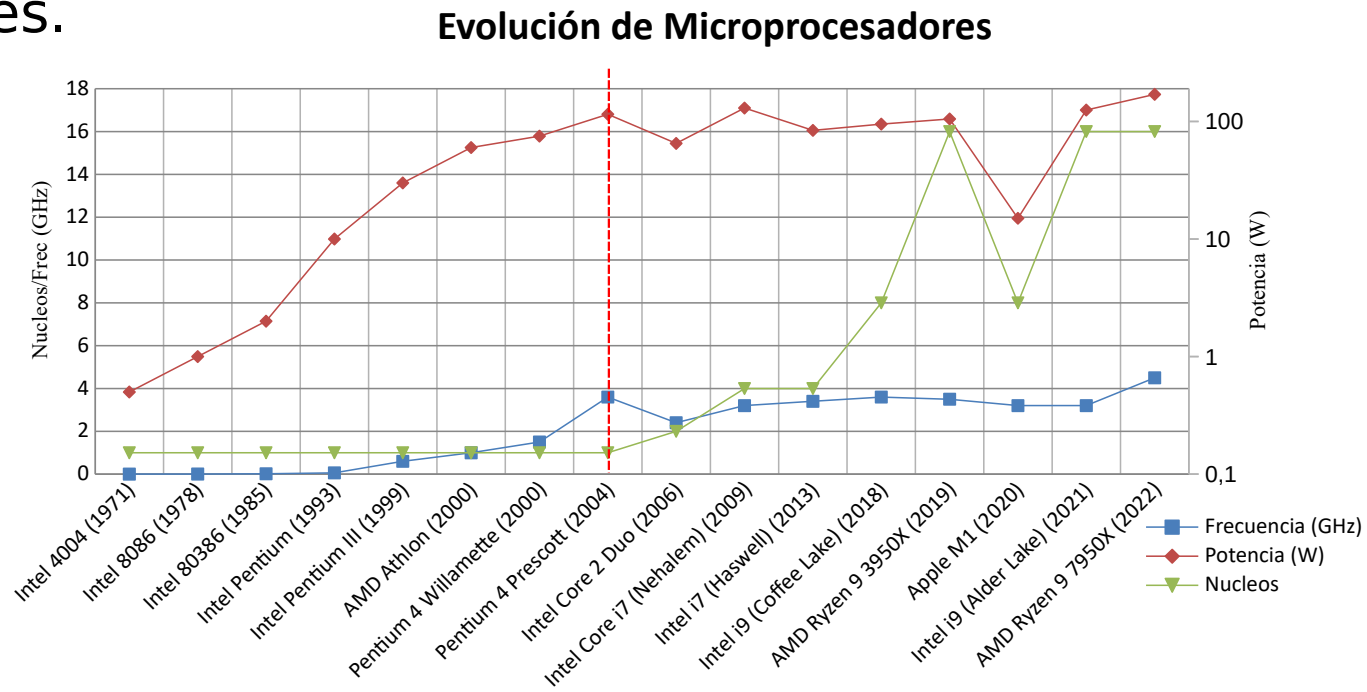
- El voltaje no puede seguir bajando → las perdidas por fugas se vuelven muy grandes
- La potencia no puede seguir subiendo → la refrigeración se vuelve impráctica



Cambio de paradigma

A partir de 2006 se cambió el rumbo, en lugar de disminuir el tiempo de respuesta de un proceso, se utilizan multiprocesadores o procesadores de varios núcleos, mejorando el rendimiento general en lugar del tiempo de respuesta de un solo proceso.

Problema: El software que antes solo recurría al avance tecnológico para correr mas rápido, ahora debe ser escrito pensando en ejecutarse en múltiples procesadores.





¿ Preguntas ?